UNIVERSITY of BRADFORD
Faculty of Engineering & Informatics

COS6012-B: Concurrent and Distributed Systems

More on Java Concurrency

Dr Daniele Scrimieri

Outline

- Previous sessions
  - `Threads; synchronized` keyword; `Lock` interface and its implementation (`ReentrantLock`); `Condition;`
  - `Semaphore` class in Java; only mentioned other utilities: `CountDownLatch, CyclicBarrier, Phaser, Exchanger, CompletableFuture`
- **Overview**
  - Thread synchronization utilities
  - Atomic variables
  - Thread pools and Executor framework
  - Fork/Join and applications
  - Concurrent collections

Other Thread Synchronization Utilities

- **`CountDownLatch`** class: mechanism that allows a thread to wait for the finalization of multiple operations.
- **`CyclicBarrier`** class: allows the synchronization of multiple threads in a common point.
- **`Phaser`** class: controls the execution of concurrent tasks divided in phases. All the threads must finish one phase before they can continue with the next one.
- **`Exchanger`** class: provides a point of data interchange between two threads.
- **`CompletableFuture`** class: provides a mechanism where one or more tasks can wait for the finalization of another task that will be explicitly completed in an asynchronous way in future.

java.util.concurrent.atomic

- Atomic classes are provided for booleans, integers, longs, and object references as well as **arrays** of integers, longs, and object references
- Most commonly used: `AtomicInteger`, `AtomicLong`, `AtomicBoolean`, and `AtomicReference` → int, long, boolean, and object reference
- **`get()`**
- **`getAndSet(`**newValue**`)`**
- **`lazySet(`**newValue**`)`**
- **`compareAndSet(`**expect, update**`)`**
- …

Example – Counter with AtomicInteger

```
public class RaceConditionAtomicInteger extends Thread {
    private static AtomicInteger total = new AtomicInteger(0);
    public static final int THREADS = 1000;
    public static final int COUNT = 10000;

    public void run() {
        for(int i = 0; i < COUNT; i++)
            total.incrementAndGet();
    }
   […]
}
```

Remember the RaceCondition from week 2 and **`synchronized void increment()`** solution?

Benefits of Using Atomic Variables

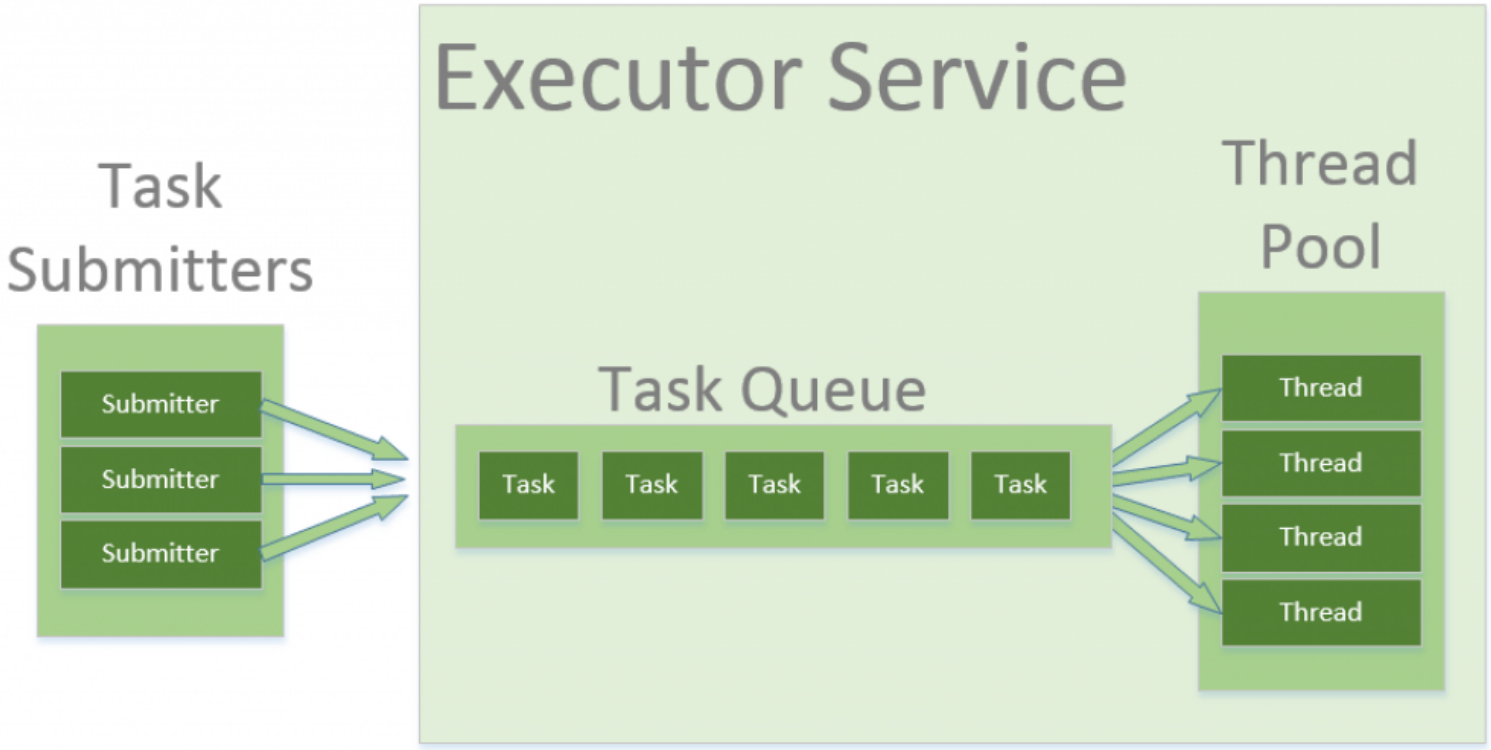
- No lock / synchronized needed
- No locks involved => it’s impossible for an operation on an atomic variable to deadlock!
- Atomic variables: foundation for non-blocking, lock-free algorithms, which achieve synchronization without locks or blocking.

Thread Pools and Executor Framework

- To develop a simple, concurrent-programming application in Java, you create some Runnable objects and then create the corresponding Thread objects to execute them.
- Developing the same way a program that runs a lot of concurrent tasks has some disadvantages:
  - You have to write all the code related to the management of the Thread objects (creation, ending, obtaining results).
  - You create a Thread object per task. If you have to execute a big number of tasks, this can affect the throughput of the application.
  - You have to control and manage efficiently the resources of the computer. If you create too many threads, you can saturate the system.

The Thread Pool

The Thread Pool pattern helps to save resources in a multithreaded application, and also to contain the parallelism in certain predefined limits.



Write code in the form of parallel tasks; submit them for execution to an instance of a thread pool.

Thread Pools and Executor Framework

- An important aspect of using the framework Executor: decoupling the tasks submission from their execution policy.
- This allows you to change the execution policy easily, without the need for major changes to the code.

```
public interface Executor {
  void execute (Runnable command);
}
```

- Different executors:
  - Executors.newCachedThreadPool()
  - Executors.newFixedThreadPool(int n)
  - Executors.newSingleThreadExecutor()

Fork/Join Framework

- It provides tools to help speed up parallel processing by attempting to use all available processor cores
- Divide and conquer approach.
- In practice, this means that the framework first “forks”, recursively breaking the task into smaller independent subtasks until they are simple enough to be executed asynchronously.
- After that, the “join” part begins, in which results of all subtasks are recursively joined into a single result, or in the case of a task which returns void, the program simply waits until every subtask is executed.
- To provide effective parallel execution, the fork/join framework uses a pool of threads called the ForkJoinPool, which manages worker threads of type ForkJoinWorkerThread.

Application: new methods in java.util.Arrays

```
public class ParallelSort {
 public static void main(String[] args) {
  List<Integer> list = new ArrayList();      Random r = new Random();
  for (int x = 0; x < 30000000; x++) {
        list.add(r.nextInt(10000));
  }
  Integer[] array1 = new Integer[list.size()];
  Integer[] array2 = new Integer[list.size()];
  array1 = list.toArray(array1);    array2 = list.toArray(array2);
  long start = System.currentTimeMillis();
  Arrays.sort(array1); long end = System.currentTimeMillis();
  System.out.println("Sort Time: " + (end - start));
  start = System.currentTimeMillis();
  Arrays.parallelSort(array2);  end = System.currentTimeMillis();
  System.out.println("Parallel Sort Time: " + (end - start));
  }
}
```

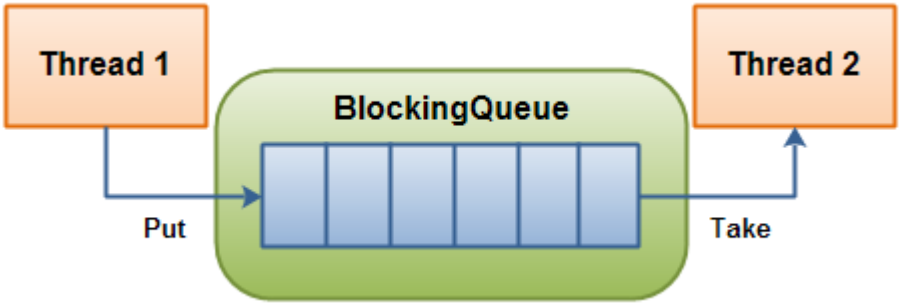
Sort Time: 14906
Parallel Sort Time: 4478

Concurrent Collections

- A key technique for properly protecting shared data is to encapsulate the synchronization mechanism with the class holding the data.
- The java.util.concurrent package holds many data structures designed for concurrent use. Generally, the use of these data structures yields far better performance than using a synchronized wrapper around an unsynchronized collection.
- **Concurrent lists and sets:** CopyOnWriteArraySet, CopyOnWriteArrayList, ConcurrentSkipListSet
- **Concurrent maps:** ConcurrentHashMap, ConcurrentSkipListMap
- **Queues:** ArrayBlockingQueue, DelayQueue, PriorityBlockingQueue, ConcurrentLinkedQueue, SynchronousQueue, LinkedBlockingQueue, LinkedTransferQueue
- **Deques:** ConcurrentLinkedDeque, LinkedBlockingDeque

Blocking Queues

- A BlockingQueue (interface) is capable of blocking the threads that try to insert or take elements from the queue.
- For instance, if a thread tries to take an element and there are none left in the queue, the thread can be blocked until there is an element to take.
- A BlockingQueue is typically used to have one thread produce objects, which another thread consumes. Implementations:
  - ArrayBlockingQueue
  - DelayQueue
  - LinkedBlockingQueue
  - PriorityBlockingQueue
  - SynchronousQueue



Blocking Queue Example

```
public class BlockingQueueExample {
  public static void main(String[] args)
      throws Exception {
    BlockingQueue queue = new ArrayBlockingQueue(1024);
    Producer producer = new Producer(queue);
    Consumer consumer = new Consumer(queue);
    new Thread(producer).start();
    new Thread(consumer).start();
    Thread.sleep(4000);
    }
}
```

```
public class Producer implements Runnable{
    protected BlockingQueue queue = null;
    public Producer(BlockingQueue queue) {
        this.queue = queue;
    }
    public void run() {
        try {
            queue.put("1");      Thread.sleep(1000);
            queue.put("2");      Thread.sleep(1000);
            queue.put("3");
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
```

This will cause the Consumer to block, while waiting for objects in the queue.

```
public class Consumer implements Runnable{
    protected BlockingQueue queue = null;
    public Consumer(BlockingQueue queue) {
        this.queue = queue;
    }
    public void run() {
        try {
            System.out.println(queue.take());
            System.out.println(queue.take());
            System.out.println(queue.take());
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

http://tutorials.jenkov.com/java-util-concurrent/blockingqueue.html

Acknowledgments (and resources for further reading)

- J. Fernandez Gonzalez (2017), Java 9 Concurrency Cookbook, Packt Publishing
- Online tutorials

http://tutorials.jenkov.com/java-util-concurrent/blockingqueue.html

https://www.baeldung.com/java-atomic-variables

https://www.baeldung.com/thread-pool-java-and-guava

https://dzone.com/articles/the-executor-framework